

FNNL — Code Review & Security Risk Assessment

Repository: github.com/bwm37/FNNL (branch: main)
Prepared for: Brian | Prepared by: Claude (Anthropic) | Date: September 1, 2026

Overall Risk Rating	LOW
Files reviewed	index.html, app.js (1,197 lines), styles.css (608 lines), README.md — 100% of repository contents
Runtime model	Static HTML/CSS/vanilla JavaScript. No build step, no package manager, no third-party runtime dependencies, no server component.
Network behavior	None found. No fetch/XHR/WebSocket calls, no CDN-hosted scripts, no remote font/image/style references.

Executive Summary

FNNL is a small, self-contained, client-side web application for building funnel charts. It is a static site made of one HTML file, one CSS file, and one JavaScript file (~1,200 lines), with no npm packages, no bundler/build step, and no server-side code. It is intended to be run locally by opening index.html directly or serving the folder with a simple static file server.

The code was reviewed line-by-line for unsafe patterns typically responsible for real-world web vulnerabilities: use of `eval/Function()`, `innerHTML/outerHTML/document.write`, dynamic script injection, unsafe URL handling, network exfiltration, prototype pollution, unsanitized use of untrusted input in the DOM, and unsafe file/CSV handling.

No unsafe code, malicious behavior, network exfiltration, or code-execution vulnerabilities were found.

The application does not make any network requests, does not use any HTML-injection sinks anywhere in its source, and builds all DOM and SVG content using safe, text-only APIs (`createElement`, `textContent`, `setAttribute`). Given that the tool is run locally and processes only data the user types or pastes into it, the practical attack surface is minimal. A small number of low-severity and informational items are noted below — mainly hardening suggestions rather than exploitable flaws — plus one item worth fixing before sharing exported CSV files with others (CSV/formula injection, Finding 1).

Findings Summary

ID	Title	Severity	Location
F1	CSV export does not neutralize spreadsheet formula triggers (CSV/formula injection)	Low	app.js — <code>makeCsv()</code> / <code>csvEscape()</code>
F2	No Content-Security-Policy set	Informational	index.html <head>
F3	Full chart configuration persisted in plaintext in browser local storage	Informational	app.js — <code>persistState()</code> / <code>loadState()</code>
F4	Legacy/imported state is only loosely type-checked before merging	Informational	app.js — <code>loadState()</code> / <code>normalizeState()</code>

F5	Deprecated document.execCommand('copy') fallback	Informational	app.js — copyCurrentCsv()
----	--	---------------	---------------------------

No High or Medium severity issues were identified. No evidence of malware, telemetry, backdoors, or obfuscated code was found anywhere in the repository.

Scope & Methodology

The entire public repository at github.com/bwm37/FNNL was retrieved and reviewed in full: `index.html` (155 lines), `app.js` (1,197 lines), `styles.css` (608 lines), and `README.md`. There are no other files, no `package.json`, no build configuration, and no CI/CD workflow files in the repository.

Review approach:

- Manual, line-by-line read-through of all four files.
- Targeted search for known-dangerous JavaScript sinks and patterns: `eval`, `new Function`, `innerHTML/outerHTML`, `document.write`, `insertAdjacentHTML`, dynamic `<script>` creation, `fetch/XMLHttpRequest/WebSocket`, `javascript: URLs`, and `dynamic import()`.
- Trace of every place user-supplied data (stage labels, variable names, chart title, pasted CSV, imported color values) flows into the DOM, into SVG output, into the exported PNG/CSV files, and into browser storage.
- Review of the CSV parser/serializer, the PNG export pipeline (SVG → `XMLSerializer` → data URL → `<canvas>` → `Blob`), and the local-storage persistence layer.
- Check for outbound network calls or references to third-party CDNs, fonts, images, or analytics/telemetry endpoints.

Application Overview

FNNL renders an interactive, multi-variable funnel chart as inline SVG, entirely in the browser. The user sets a stage count (2–10) and a variable count (1–10), fills in a values matrix, optionally imports/pastes CSV data, and can export the current chart as a PNG (via an offscreen `<canvas>`) or export/copy the current data as CSV. The last-used configuration is saved to `localStorage` under the key `fnnl-state-v2` (with a fallback read of a legacy key from an earlier version of the app) so the chart is restored on the next visit.

There is no server, no authentication, no user accounts, and no data ever leaves the browser tab. This matches the README's description of the tool as a "local, dependency-free web application."

Positive Security Observations

- **No dangerous DOM sinks anywhere in the code.** A full-text search of `app.js` for `innerHTML`, `outerHTML`, `document.write`, `eval`, `new Function`, and `insertAdjacentHTML` returns zero matches. Every DOM and SVG node is built with `document.createElement/createElementNS`, and every piece of user-typed text (stage labels, variable names, chart title, imported CSV values) is written using `textContent` or the DOM text-node API, never parsed as markup. This structurally rules out DOM-based XSS from anything a user types or pastes.
- **No network activity of any kind.** There are no calls to `fetch`, `XMLHttpRequest`, `WebSocket`, `sendBeacon`, or any remote URL. `index.html` loads only its own local `styles.css` and `app.js` — no CDN scripts, web fonts, images, or analytics snippets. The app cannot phone home or exfiltrate chart data even if it wanted to.
- **Zero third-party dependencies.** There is no `package.json`, no `node_modules`, and no bundler. This eliminates the supply-chain risk (compromised npm package, malicious transitive dependency, etc.) that affects most modern web apps.

- **Careful input coercion.** Numeric fields are parsed defensively (`parseFloat`, `clampInt`, `clampNumber`) with `Number.isFinite` checks, so malformed or extreme input (e.g. pasted CSV garbage) degrades to safe defaults (0, or a clamped min/max) rather than producing NaN/Infinity that could corrupt rendering or storage.
- **Strict color validation.** `normalizeHex()` only accepts well-formed 3- or 6-digit hex colors via regex and otherwise silently substitutes a safe fallback color — untrusted color strings can't reach a CSS/SVG `url()` or attribute-injection context.
- **Hand-rolled CSV parser, not eval-based.** `parseCsv()` is a small character-by-character state machine; it does not use `eval`, regex-based dynamic code, or a third-party parsing library.
- **Self-contained export pipeline.** PNG export serializes the on-screen SVG with `XMLSerializer`, loads it into an `Image` via a same-document `data: URL`, draws it to a `<canvas>`, and saves it via a `Blob/URL.createObjectURL` download link. No external image or font is fetched during export.
- **No inline event handler attributes.** `index.html` contains no `onClick=/onerror=` style attributes; all interactivity is wired up in `app.js` via `addEventListener`, which is both cleaner and slightly more CSP-friendly.

Detailed Findings

Finding 1 — CSV export does not neutralize spreadsheet formula triggers

Low

Location: app.js — makeCsv() (line ~989) and csvEscape() (line ~1042)

Description: The CSV export/copy/download feature writes variable names, colors, and stage labels straight into CSV cells. csvEscape() only wraps a cell in quotes and doubles internal quotes when the cell contains a comma, quote, or newline — it does not check for a leading =, +, -, or @ character. If a user (or CSV pasted from an untrusted source) puts a value like =cmd | '/c calc' !A1 into a variable name or stage label, that exact string will be written unmodified into the downloaded .csv file.

```
function csvEscape(value) {
  let text = String(value ?? '');
  if (/^[=+\-@\t\r]/.test(text)) text = `'$${text}`; // neutralize formula triggers
  return /[",\n\r]/.test(text) ? `"$${text.replace(/"/g, '""')}"` : text;
}
```

Practical impact: This is the classic "CSV/Formula Injection" issue (CWE-1236). It has no effect inside this app itself — FNNL never re-parses its own export as a formula — but if the exported CSV file is later opened in Excel, Google Sheets, or LibreOffice Calc (which is the CSV's whole purpose), that spreadsheet program may interpret the leading character as a formula and execute it, potentially triggering external commands, the DDE/exec exploit chain, or automatic outbound requests. Since values typically come from the same user's own typed input, the realistic scenario is limited to: (a) a user pastes a spreadsheet exported by someone else with hostile column headers/names and re-exports it, or (b) an exported CSV is shared with a colleague who opens it without caution.

Recommendation: Prefix any cell whose first character is =, +, -, @, tab, or carriage return with a leading single quote (') before the existing quoting logic runs, so spreadsheet programs treat the cell as plain text. This is a few lines in csvEscape() and is the standard, widely-recommended mitigation for this class of issue.

Finding 2 — No Content-Security-Policy configured

Informational

Location: index.html — <head>

Description: index.html sets no Content-Security-Policy meta tag and, being a plain static file, sends no CSP response header either. Today this has no practical exploit path — the review found no innerHTML/eval usage for a CSP to guard — but a CSP acts as a safety net if a future code change accidentally introduces one of those sinks.

Practical impact: None currently exploitable. This is defense-in-depth for future maintenance, not a fix for an existing hole.

Recommendation: Optionally add a restrictive meta CSP, e.g. <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:; connect-src 'none'">, which matches the app's existing fully-local, no-network design and would make any accidental future regression immediately fail closed.

Finding 3 — Chart configuration is persisted in plaintext in browser localStorage

Informational

Location: app.js — persistState() and loadState()

Description: The entire application state — chart title, stage labels, variable names/colors/values, theme, etc. — is serialized with JSON.stringify and written to localStorage under the key fnnl-state-v2 so the

last chart reloads automatically. This is convenient, expected behavior for a local tool, and is not a vulnerability in the code, but it does mean funnel data typed into the app remains readable on disk (via the browser's local storage files) after the tab is closed.

Practical impact: Low. If the browser profile is shared by multiple people on the same machine, the most recently entered funnel data (which the README implies may be business metrics such as impressions/add-to-cart/purchase figures) will be visible to the next person who opens the app in that browser profile, until "Clear values" is pressed or storage is cleared.

Recommendation: No code change is required for the tool's stated purpose. If the data is sensitive, use a private/incognito browser window (which discards localStorage on close) or add a manual "Clear saved data" control that also removes the localStorage key, in addition to the existing "Clear values" button which only zeroes the value matrix.

Finding 4 — Imported/legacy state is only loosely validated before use

Informational Location: app.js — loadState() (line ~1055) feeding normalizeState() (line ~265)

Description: loadState() reads either the current or a legacy localStorage key, JSON.parse() it, and accepts the result as long as it is a non-null object — it does not check the shape of nested fields before merging. The immediately following normalizeState() does coerce every field it actually uses (String(...), clampInt(...), array length-trimming, hex validation, etc.), which is why this does not currently translate into an exploitable issue.

Practical impact: None currently exploitable — every field that reaches the DOM or SVG output is coerced to a safe primitive type first. This is only worth tracking because it is the one place in the codebase where trust in the shape of input is looser than everywhere else, so future edits to normalizeState() should preserve that coercion rather than remove it.

Recommendation: No change required today. If normalizeState() is ever refactored, keep explicit type coercion (not just presence checks) on every field read from stored/imported state before it is used.

Finding 5 — Deprecated document.execCommand('copy') fallback

Informational Location: app.js — copyCurrentCsv() (line ~962)

Description: The "Copy current CSV" button first tries the modern async Clipboard API (navigator.clipboard.writeText) and, if that throws, falls back to selecting the CSV textarea and calling the deprecated document.execCommand('copy').

Practical impact: None. This is a functionality/compatibility note, not a security issue — execCommand('copy') only copies text the app itself already placed in a visible, focused textarea that the user can inspect; it cannot read or write anything outside that field.

Recommendation: No action required. Optional: drop the fallback once the target browsers can be assumed to support the async Clipboard API, purely for code simplicity.

Conclusion & Recommendations

FNNL is a small, well-written, dependency-free client-side tool. Across a full read of every line of `index.html`, `app.js`, and `styles.css`, no unsafe code, remote code execution paths, DOM-based XSS sinks, network exfiltration, obfuscation, or other malicious behavior was identified. Given that it is run locally by opening a file or serving it from a local static server — with no accounts, no server-side logic, and no data leaving the browser — the practical risk of running this application locally is low.

Suggested priority order if any changes are made:

- **Fix Finding 1 (CSV/formula injection)** before sharing exported CSV files with other people or other spreadsheet tools — this is the only item with a realistic, if narrow, real-world impact.
- Treat Findings 2–5 as optional hardening / cleanup; none block safe local use of the app as it exists today.

This assessment is a manual source-code review of the repository contents as of the date above and does not constitute a formal penetration test or automated static/dynamic security scan.